

Chapter 16

ggplot2

by Lorraine Gaudio

Version May 29, 2026

Contents

1	Overview	2
2	Setting Up Your Notes	2
2.1	Create a Quarto File	2
2.2	YAML Header	2
2.3	Headings	3
3	Packages	3
4	What is ggplot2?	3
5	The Blank Canvas	4
6	Mapping Aesthetics	5
7	Adding Geometric Objects (Geoms)	6
7.1	Scatterplot Example	6
7.2	Layering geoms	8
7.3	Histogram Example	9
7.4	Density Plot	12
7.5	Overlaid Histograms	13
7.6	Boxplot	14
8	Labels and Themes	15
9	Summary	16
10	Chapter Terms	17
11	Practice Space	17
11.1	Setup	17
11.2	Task 0	18
11.3	Task 1	19
11.4	Task 2	19
11.5	Task 3	19
11.6	Task 4	19
11.7	Task 5	20
11.8	Task 6	20
11.9	Save and Upload	21

1 Overview

In earlier chapters, you practiced working with data frames, variables, and reproducible Quarto notes. In this chapter, you will use `ggplot2` to turn data into graphs that are easier to read and explain. You will build plots by combining a data frame, aesthetic mappings, and geometric layers.

This chapter matters because analysts do not only calculate results. They also need to communicate patterns, comparisons, distributions, and possible relationships in a clear visual form. In R, `ggplot2` is one of the main tools for doing that.

By the end of Chapter 16 you will be able to:

- Build a plot by combining `data`, `aes()`, and `geom_*()`.
- Map variables to visual properties such as the x-axis, y-axis, `fill`, and `color`.
- Create scatterplots with `geom_point()`.
- Create histograms with `geom_histogram()`.
- Create boxplots with `geom_boxplot()`.
- Improve readability with labels and themes.
- Continue using Quarto to organize code, output, checks, and written interpretation in a reproducible analysis document.

Keep these goals in mind as you move through each section.

2 Setting Up Your Notes

In Chapter 11, you learned how to create a Quarto file, keep the YAML header, use Markdown headings, write memos outside code chunks, and control chunks with `#|` options. In Chapter 12, you added a table of contents and section numbering. In Chapter 13, you continued using those Quarto skills in a reproducible workflow. In Chapter 16, you will keep practicing those same habits while learning `ggplot2`.

2.1 Create a Quarto File

For those of you who **have GitHub** and a course project, navigate to your course folder and open your course project. This will open RStudio. Click “Pull” in the Git pane to get the latest version of the course materials. After completing work, save, stage, commit, and push.

If you **don’t have GitHub** or a course project, just open RStudio and set your working directory to your course folder.

Open a new Quarto document in your course folder and save it as: `chapter16_notes.qmd` (File > New File > Quarto Document).

2.2 YAML Header

Copy and paste this YAML header into your new Quarto document. Update the author field with your name.

```
---
title: "Chapter 16 Notes"
subtitle: "ggplot2"
author: "YOUR NAME"
date: today
toc: true
toc-depth: 2
number-sections: true
format:
  html:
```

```
code-fold: false
```

Since this document can act as a resource for you to review later, set `code-fold` to `false` so that all code is visible by default.

2.3 Headings

Use the section headings in this file to structure your notes. You can copy and paste these into your new Quarto document:

```
# Loading Packages
# What is ggplot2?
# The Blank Canvas
# Mapping Aesthetics
# Scatterplots
# Histograms
# Boxplots
# Labels and Themes
```

Work through the sections under each heading. Render to HTML often to check both formatting and code output.

3 Packages

Install once (if needed): `install.packages("dplyr"); install.packages("dslabs")`

```
install.packages("ggplot2")
```

Turn off `install.packages()` after you have installed the package once. You only need to install a package one time on your computer. After that, you can load the package with `library()`.

Load the packages at the start of every session:

```
#
library(dslabs) # Data science labs package
library(dplyr)  # Data manipulation package
library(ggplot2) # Data visualization
```

4 What is ggplot2?

Open the help file for `ggplot()` to see its formal arguments and description.

```
?ggplot # skim the Details section
```

The SYNTAX (basic/general):

```
ggplot(data = NULL, mapping = aes(), ..., environment = parent.frame())
```

The three core steps of **ggplot2** are: (1) supply a data frame, (2) map aesthetics with `aes()`, and (3) add geometric objects (geoms) with `geom_*()` functions.

5 The Blank Canvas

We'll start building a plot step-by-step. First, we create a blank canvas with `ggplot()`. The data set we'll use is `divorce_margarine` from the `dslabs` package.

```
data("divorce_margarine") # Load the divorce_margarine data
View(divorce_margarine)   # View the data frame
```

Goal. Start every plot by piping a data frame into `ggplot()`.

```
df %>% ggplot()
# Code Example: Blank canvas
divorce_margarine %>%
  ggplot()
```

```
# Without piping:
ggplot(data = divorce_margarine)
```

Comment: The result is an empty canvas.

R does not throw an error. Note that `ggplot` needs three main steps (`data` → `aes()` → `geom_*()`).

6 Mapping Aesthetics

Aesthetic mapping is the process of connecting variables in your data to visual properties of the plot, such as position, color, size, and shape. In `ggplot2`, this is done using the `aes()` function.

Review the syntax for `ggplot2` again.

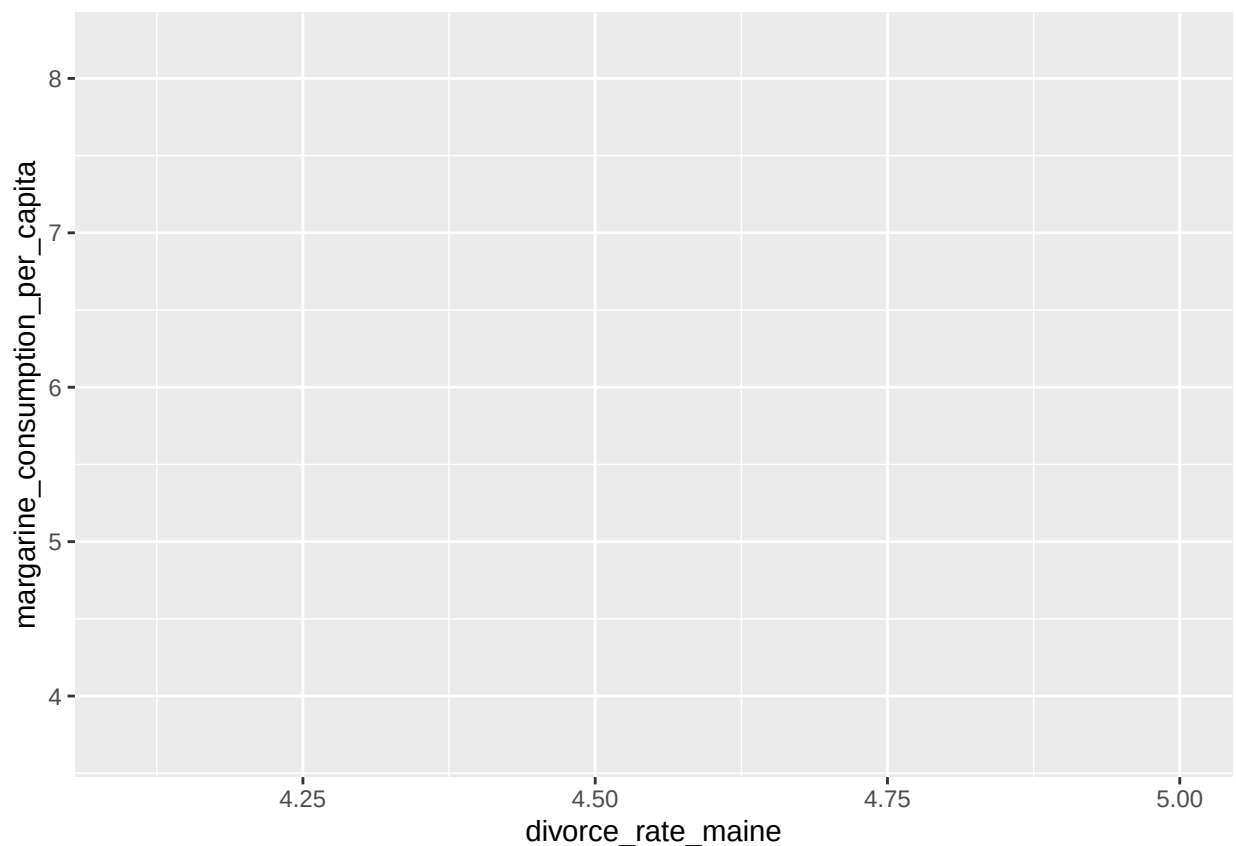
The SYNTAX (basic): `ggplot(data = df, mapping = aes(x, y, other aesthetics))`

The `aes()` argument supplies mapping information. Mapping refers to connecting your data variables to visual properties (aesthetics) of the plot. It's how you tell `ggplot` which column in your data should control which aspect of the visualization.

```
?aes
```

The GOAL: Map divorce rate to the x-axis and margarine consumption to the y-axis using `aes()`.

```
# Code Example: Mapping aesthetics
divorce_margarine %>%
  ggplot(aes(x = divorce_rate_maine, y = margarine_consumption_per_capita))
```



Comment: The plot is still blank because we haven't added any geometric objects (geoms) yet. The axes are now labeled by the mapped variables, even if there are no points yet. `aes()` connects data columns to visual positions. The `aes()` step comes before choosing a geom because we are first defining the mapping (what goes on the x- and y-axes, colors, etc.) and then deciding how to draw those mapped variables (points, lines, bars, etc.).

Reflection: The aesthetics describe the underlying structure of the plot, and the geoms are just different ways of rendering that structure. By setting `aes()` inside `ggplot()` we create a shared mapping that all later geoms can reuse, instead of repeating ourselves for each layer.

7 Adding Geometric Objects (Geoms)

Geometric objects (**geoms**) are the actual shapes drawn on the plot: points, lines, bars, boxplots, etc. Each geom answers the question: *Given these aesthetics, how should the data be drawn?*

7.1 Scatterplot Example

Scatterplots are used to visualize the relationship between two numeric variables. Each point represents an observation in the data, with its position determined by the values of the two variables.

`geom_point()` is a geometric object (or “geom”) in `ggplot2` that creates a scatter plot by placing points at the specified x and y coordinates.

```
?geom_point
```

The SYNTAX:

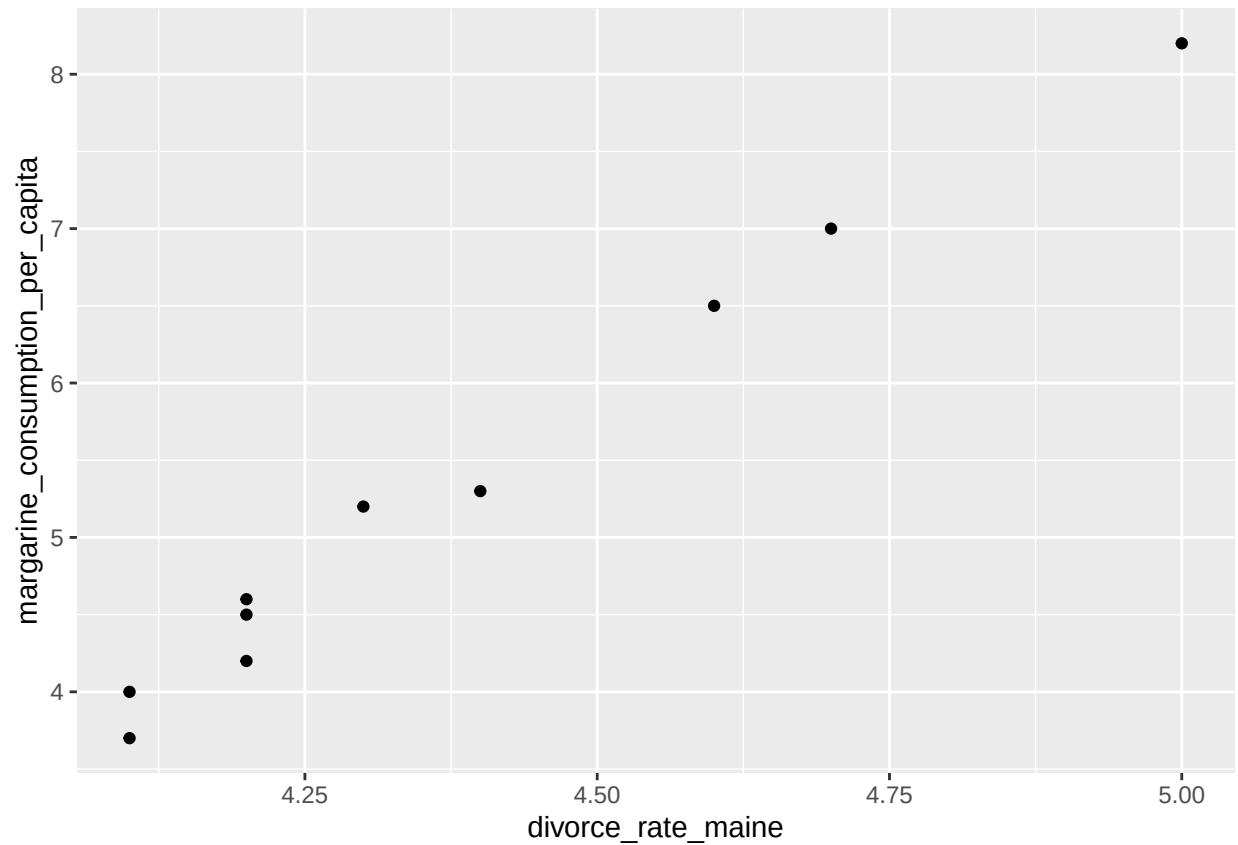
```
geom_point(mapping = NULL, data = NULL, ..., na.rm = FALSE, show.legend = NA, inherit.aes = TRUE)
```

Notice that all of the arguments are have default settings because `geom_point()` can inherit the data and aesthetic mappings from the initial `ggplot()` call.

When using `ggplot2`, geoms are added to the plot using the `+` operator.

1. The GOAL: Add points to the plot using `geom_point()`.

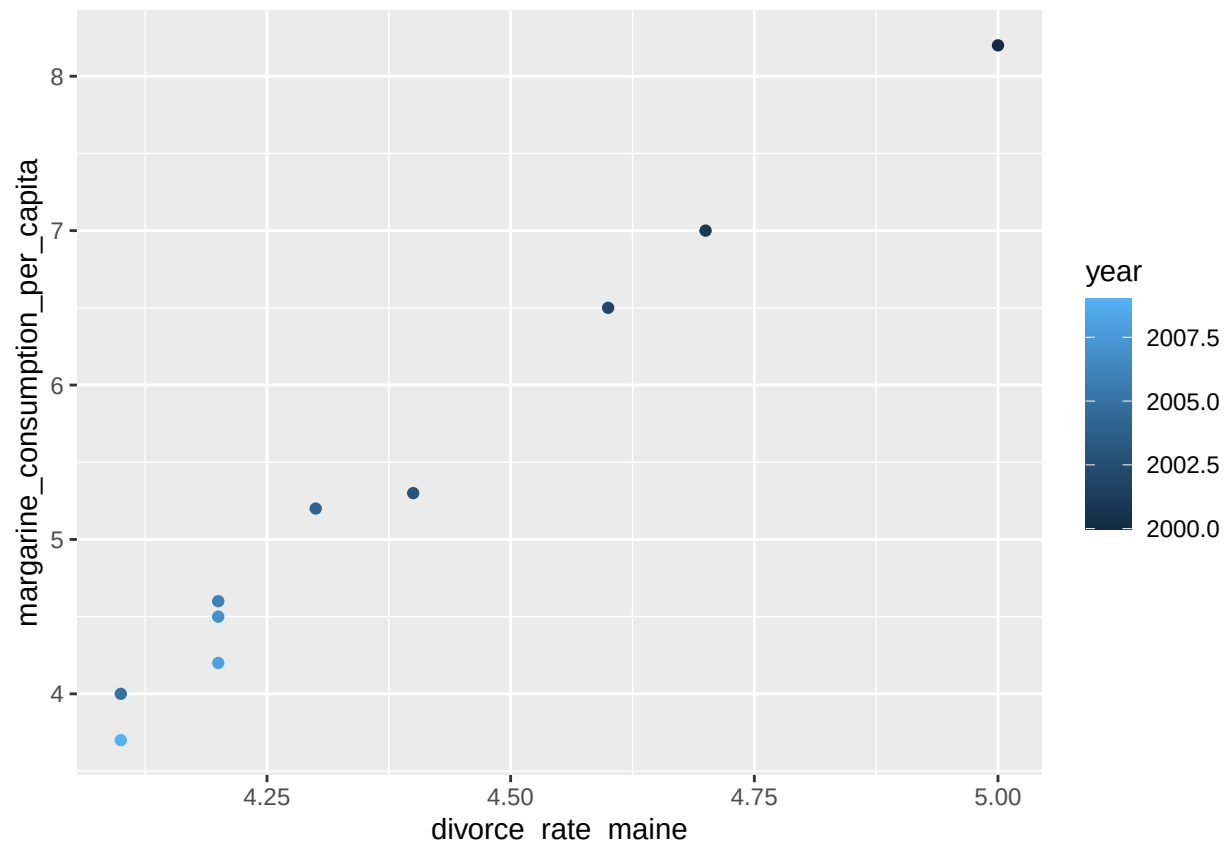
```
# Code Example: Scatterplot
divorce_margarine %>%
  ggplot(aes(divorce_rate_maine, margarine_consumption_per_capita)) +
  geom_point()
```



Comment: There are points showing a linear, positive association between divorce rate and margarine consumption per capita.

2. The GOAL: Map additional aesthetics inside `geom_point()`: color by year, size by divorce rate, and transparency (alpha) by margarine consumption.

```
divorce_margarine %>%
  ggplot(aes(x = divorce_rate_maine, # Global aes mapping: x-axis
             y = margarine_consumption_per_capita)) + # Global aes mapping: y-axis
  geom_point(aes(color = year)) # Local aes mapping: color points by year
```



“Wait”, you might be thinking, “`aes()` is already defined in `ggplot()`, why are we using it again in `geom_point()`?” The `aes()` in `ggplot()` are global mappings for the entire plot and apply to all layers. When we use `aes()` inside a specific geom like `geom_point()`, we’re defining mappings that are specific to that layer. This allows for more granular control over how different layers of the plot behave.

`geom_point()` understands these aesthetics:

- x, y: Position (required)
- alpha: Transparency
- color: Point color
- fill: Fill color (for shapes with fill)
- shape: Point shape (0-25)
- size: Point size
- stroke: Border thickness

Explore and Play: Try adding any of the above aesthetics to `geom_point()`. How does it change the plot?

7.2 Layering geoms

Different geoms tell different stories. Let’s explore more geoms!

We’ve seen that `geom_point()` highlights individual data points. `geom_smooth()` adds a smoothed line to show trends in the data.

```
?geom_smooth
```


The SYNTAX:

```
geom_smooth(mapping = NULL, data = NULL, method = NULL, se = TRUE, ...)
```

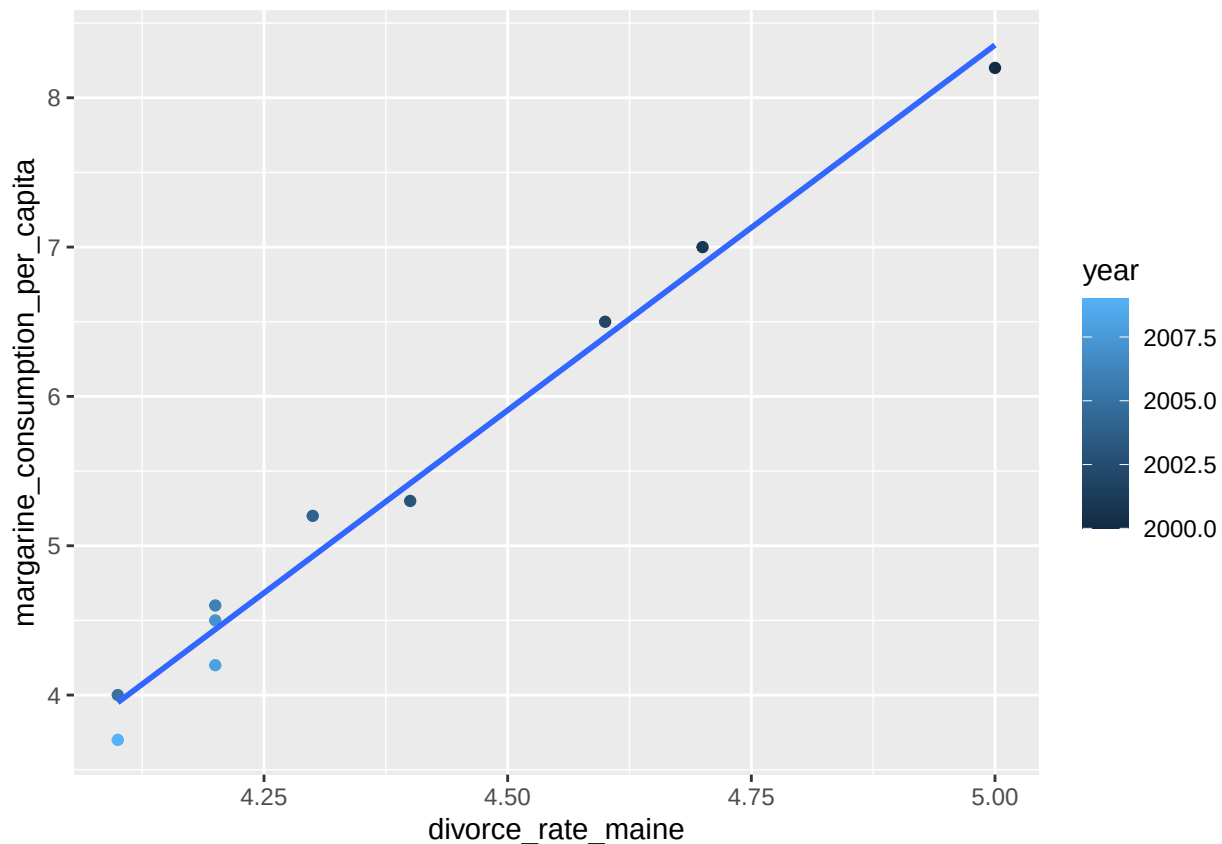
Look at the argument `method`: This controls the smoothing method used. Common options include:

- “lm”: Linear model (straight line)
- “loess”: Local regression (curved line)

The GOAL: Add `geom_smooth()` to the scatterplot.

Since the relationship looks linear, we’ll use `method = "lm"` to fit a straight line. We’ll also set `se = FALSE` to hide the confidence interval around the line for clarity.

```
# Code Example: Scatterplot with trend line
divorce_margarine %>%
  ggplot(aes(x = divorce_rate_maine,
             y = margarine_consumption_per_capita)) +
  geom_point(aes(color = year)) +           # Scatter points
  geom_smooth(method = "lm", se = FALSE)    # Add linear trend line
```



Comment: The trend line shows a positive linear relationship between divorce rate and margarine consumption.

7.3 Histogram Example

Histograms are used to visualize the distribution of a single numeric variable. They group values into bins and show the count (or density) of observations in each bin.

```
?geom_histogram
```

The SYNTAX:

```
geom_histogram(mapping = NULL, data = NULL, ..., binwidth = NULL, bins = NULL, na.rm = FALSE, show.legend = NA, inherit.aes = TRUE)
```

Let's build our own data set to explore histograms.

1. The GOAL: Simulate a **Tallness** data frame with heights for 1,000 people in the USA and Canada each. In the column **Country**, make two groups labeled "USA" and "CAN". USA heights are centered around 67 inches with a standard deviation of 2.5. CAN heights are centered around 66 inches with a standard deviation of 1.5. Tallness follows a normal distribution.

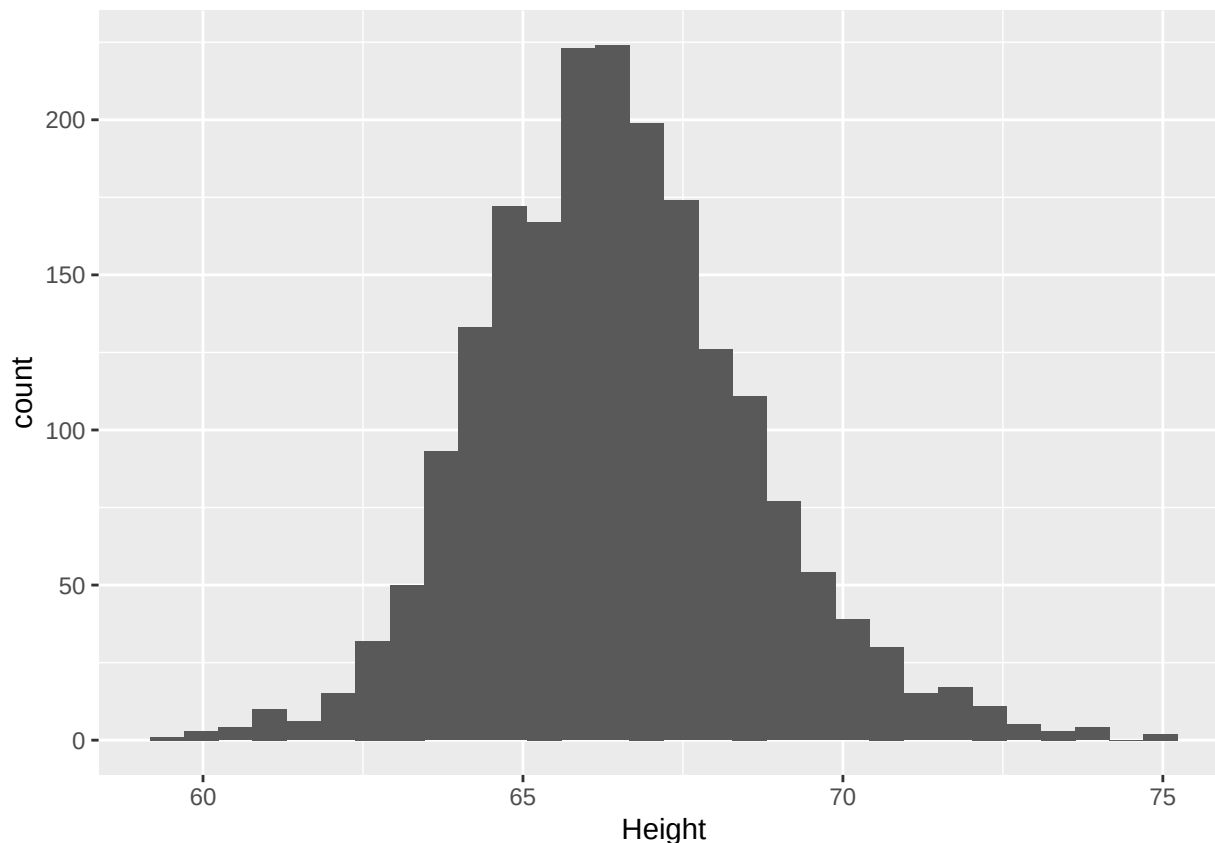
```
# Code
set.seed(2025)
Tallness <- data.frame(
  Country = c(rep("USA", 1000), rep("CAN", 1000)),
  Height  = c(rnorm(1000, 67, 2.5), rnorm(1000, 66, 1.5))
)
```

```
# Verify Tallness data frame
nrow(Tallness)
Tallness %>%
  group_by(Country)%>%
  summarize(n = n(),
            mean_height = mean(Height),
            sd_height = sd(Height),
            .groups = "drop")
```

Comment: We have 2,000 rows, 1,000 for each of the two country labels, and heights roughly around expected means and standard deviation.

2. The GOAL: Build a histogram of heights using `geom_histogram()`.

```
# Code Example: Histogram
Tallness %>%
  ggplot(aes(Height)) + # only x for histograms
  geom_histogram()
```



Comment: Histograms visualize the distribution of a single continuous variable. `geom_histogram()` uses the global `x` aesthetic (`Height`) and so there is no need to repeat the `aes()` inside of the `geom_histogram()` argument. Additionally, the `y` is computed, not mapped. `geom_histogram()` uses `stat_bin()` “under the hood” to compute the counts (or density). This is why we do not need to map a column to `y`. The `y`-values are produced by the statistic.

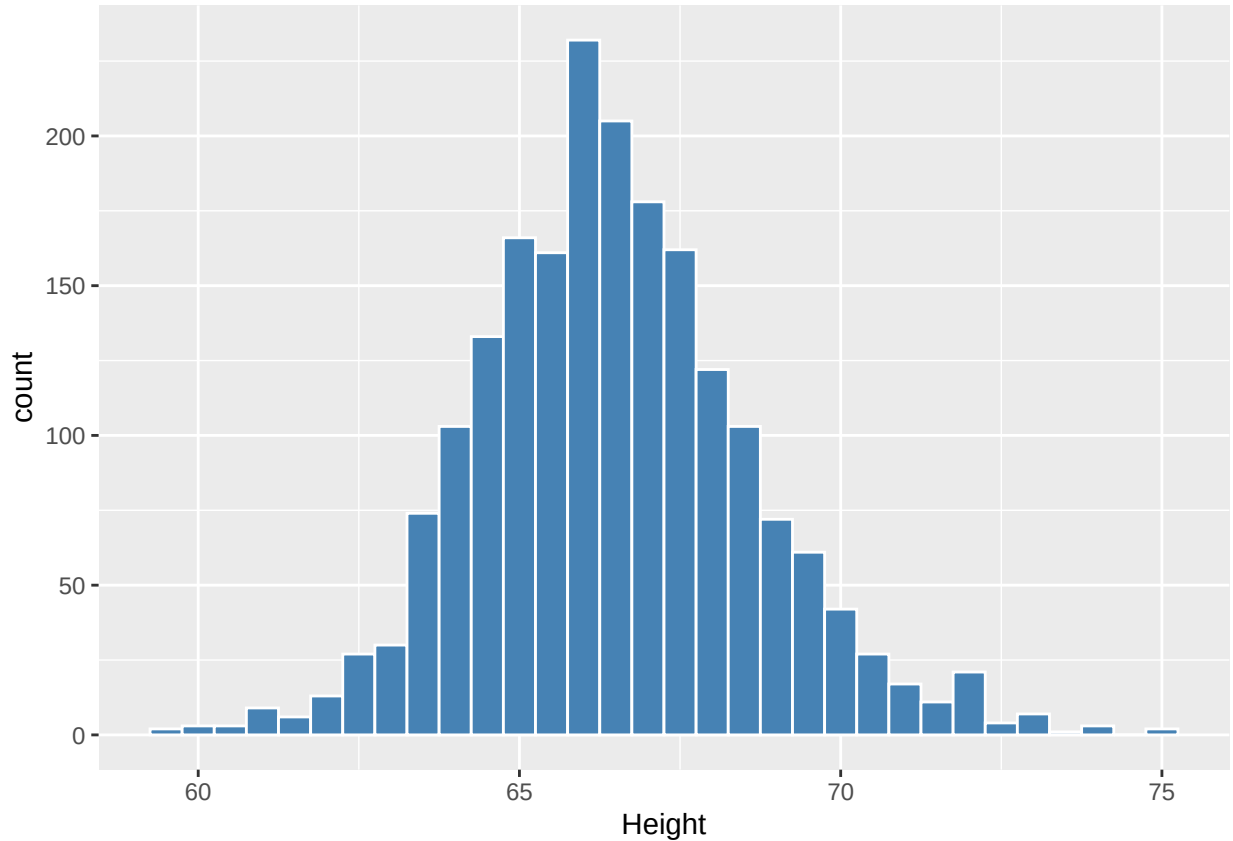
`geom_histogram()` understands these aesthetics:

- `x`: Position (required) *and usually entered in `ggplot(aes())`*
- `fill`: Fill color
- `color`: Border color
- `binwidth`: Width of each bin (default is 1)
- `position`: Position adjustment (e.g., “identity”, “dodge”, “fill”)
- `alpha`: Transparency
- `size`: Border thickness
- `weight`: Weight for each observation (optional)
- `linetype`: Line type for borders (optional)

3. The GOAL: Customize the histogram by setting `fill`, `color`, and `binwidth`.

```
# Code Example: Customized Histogram
Tallness %>%
  ggplot(aes(Height)) +           # only x for histograms
  geom_histogram(fill = "steelblue",
```

```
color = "white",      # white border gives gap between bars
binwidth = 0.5)      # Set bin width to 0.5
```



Comment: The histogram shows a unimodal distribution roughly centered in the high 60s.

Explore and Play: Change `binwidth = 0.25` inside `geom_histogram()`. Are the overall center and spread the same, even though the “smoothness” changes? Reflect on how binwidth trades off smooth vs. noisy appearance and what you’d choose for a report.

Break Things! Test out your hand at adding aesthetics `geom_histogram()` understands.

7.4 Density Plot

Now we’ll look at a smooth version of the height distribution using a density curve.

Density plots are smoothed versions of histograms that estimate the probability density function of a continuous variable. They are useful for visualizing the distribution shape without the binning artifacts of histograms.

```
?geom_density
```

The SYNTAX:

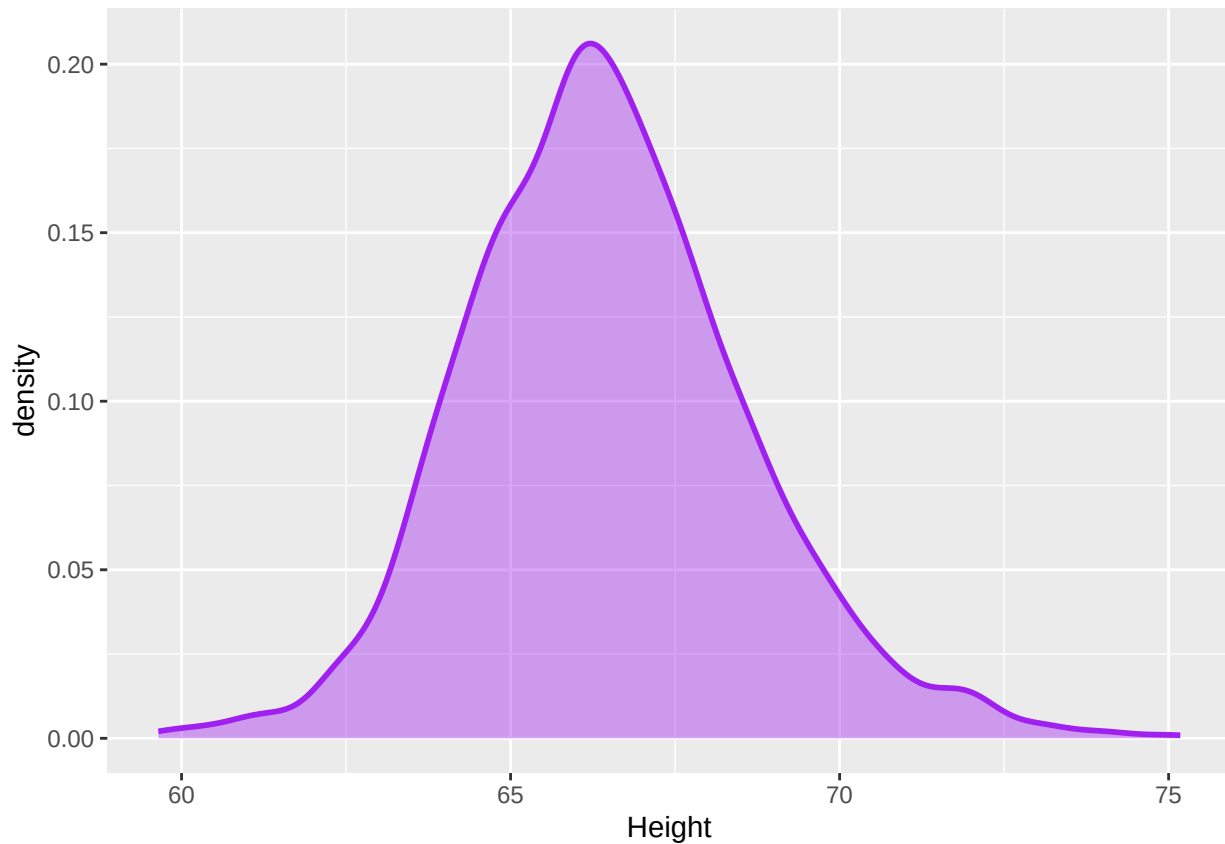
```
geom_density(mapping = NULL, data = NULL, ..., na.rm = FALSE, show.legend = NA, inherit.aes = TRUE)
```

The GOAL: Make a smooth density plot of heights in the **Tallness** data and emphasize that the vertical axis is density, not counts. Add a density layer that shows a smooth curve for the distribution of heights. Fill the area under the curve, make it partially transparent so the shape is not too heavy.

```
# Code Example: Density Plot
```

```
Tallness %>%
```

```
  ggplot(aes(x = Height)) +  
    geom_density(fill = "purple",  
                 alpha = 0.4,  
                 color = "purple",  
                 linewidth = 1)
```



`geom_density()` uses kernel density estimation to create a smooth curve that represents the distribution of the data.

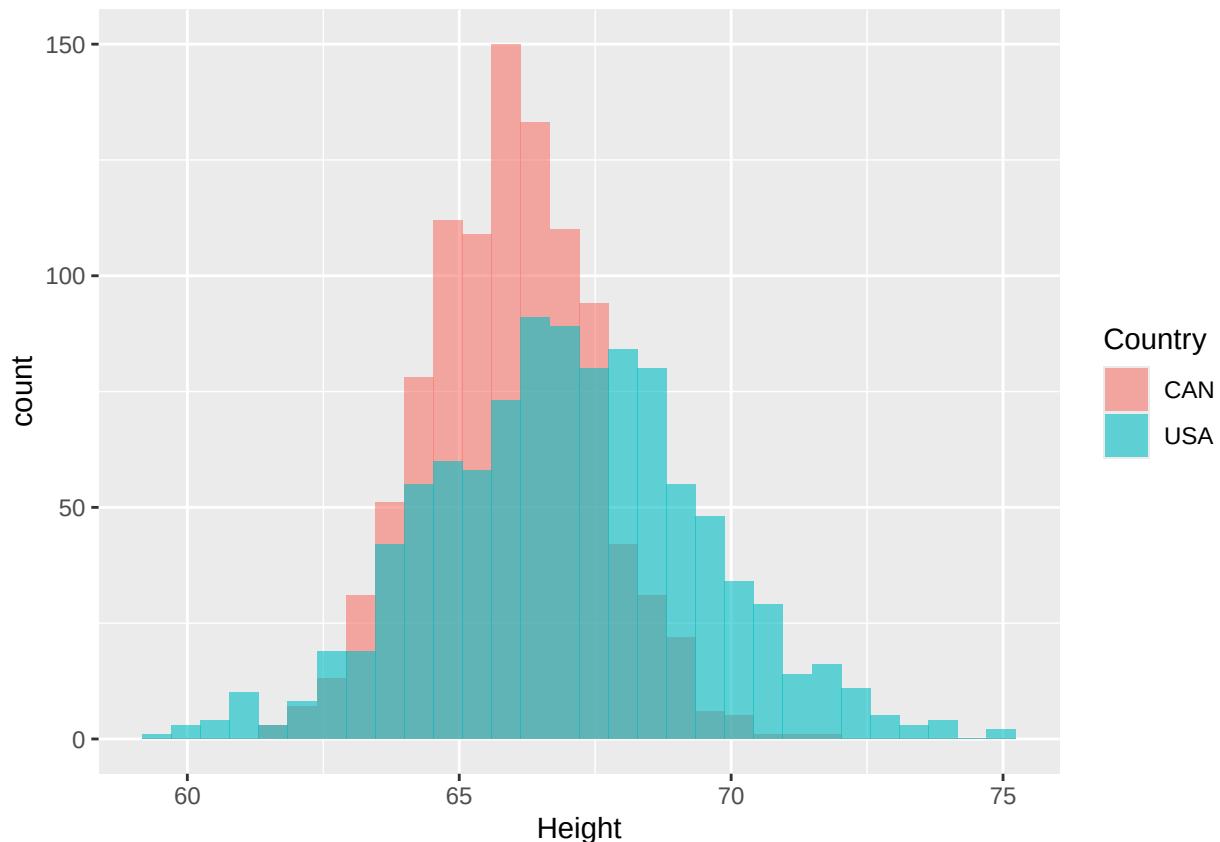
- `x` is the variable whose distribution you want to smooth and is usually entered in `ggplot(aes())`.
- `geom_density()` computes the vertical axis as density by default; the y-values are not mapped from a column.
- `fill` sets the interior color of the density shape.
- `alpha` controls transparency (0 = fully transparent, 1 = solid).
- `color` sets the outline color.
- `linewidth` controls the thickness of the outline.

7.5 Overlaid Histograms

Let's compare the two height distributions by overlaying histograms and using `fill = Country` to distinguish groups.

The GOAL: Compare height distributions between USA and CAN by overlaying two histograms using `fill = Country`. Let's store the graph as `tall_compared_fig`. Then, we'll print the figure to take a look.

```
# Code Example: Overlaid Histograms
tall_compared_fig <- Tallness %>%
  ggplot(aes(Height, fill = Country)) +
  geom_histogram(position = "identity",
                 alpha = 0.6) # Set transparency to see overlap
# Print the figure
print(tall_compared_fig)
```



Comment: `fill = Country` in the global `aes()` tells `ggplot()` to make a separate histogram for each country with different fill colors, and automatic a legend. By default, `geom_histogram()` stacks groups. The `position = "identity"` instead puts the bars in the same x-scale so they overlap. The `alpha = 0.6` makes bars transparent (0 = invisible, 1 = solid), so we can see where the two distributions overlap.

7.6 Boxplot

Boxplots summarize distributions using five statistics: minimum, first quartile (Q1), median, third quartile (Q3), and maximum. They are useful for comparing distributions across groups.

```
?geom_boxplot
```

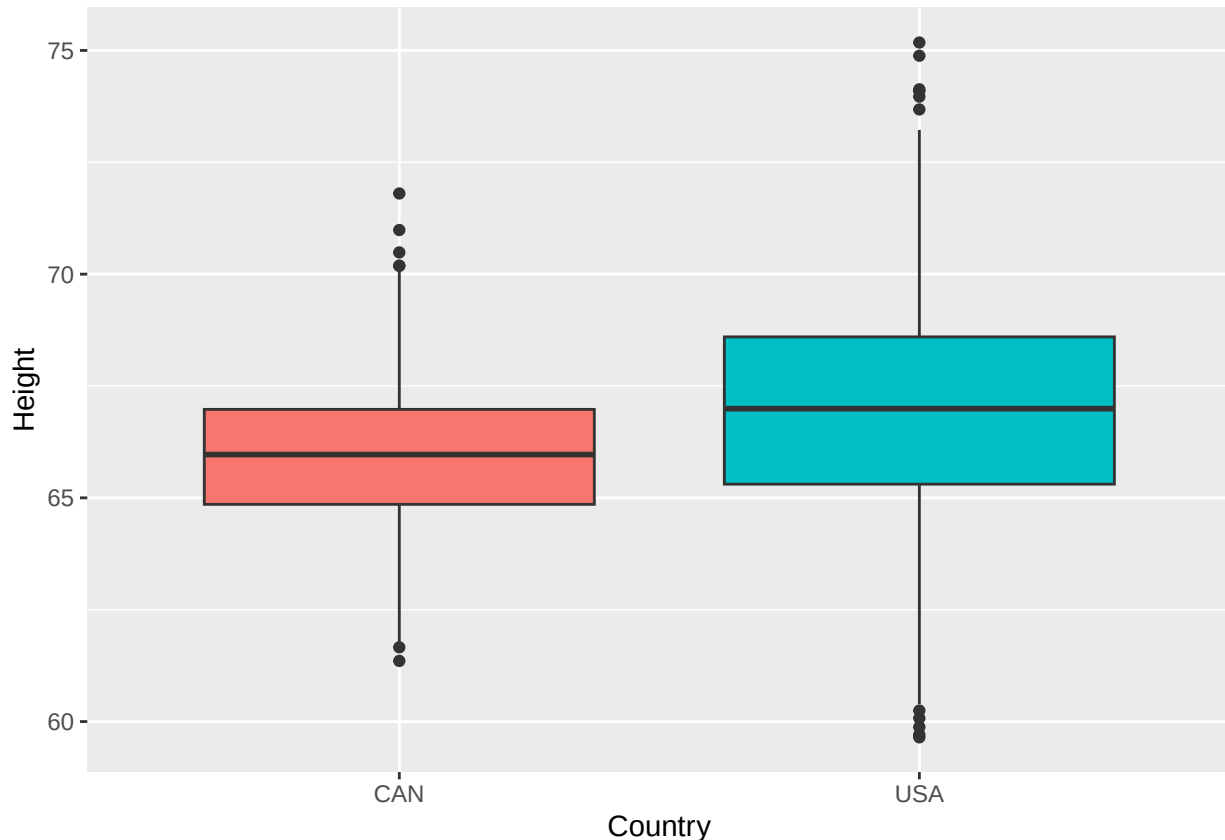
The SYNTAX:

```
geom_boxplot(mapping = NULL, data = NULL, ..., outlier.color = NULL, outlier.shape = 19,
             outlier.size = 1.5, na.rm = FALSE, show.legend = NA, inherit.aes = TRUE)
```

We can use boxplots to compare height distributions between USA and CAN.

The GOAL: Build side-by-side boxplots of height by country using `geom_boxplot()`. Map `Country` to the x-axis and `Height` to the y-axis. Use `fill = Country` to color the boxes by country and set `show.legend = FALSE` to hide the legend. We'll store the boxplot as `Tall_Box`.

```
# Code Example: Boxplot
Tall_Box <- Tallness %>%
  ggplot(aes(Country, Height, fill = Country)) +
  geom_boxplot(show.legend = FALSE)
# Print the boxplot
Tall_Box
```



Comment: There are two boxes, one for each country, with USA maybe a bit higher median and wider spread. The countries appear on the x-axis and that each box shows median, IQR, and dots for each of the outliers.

8 Labels and Themes

We can enhance our plots by adding axis labels and applying themes for better aesthetics.

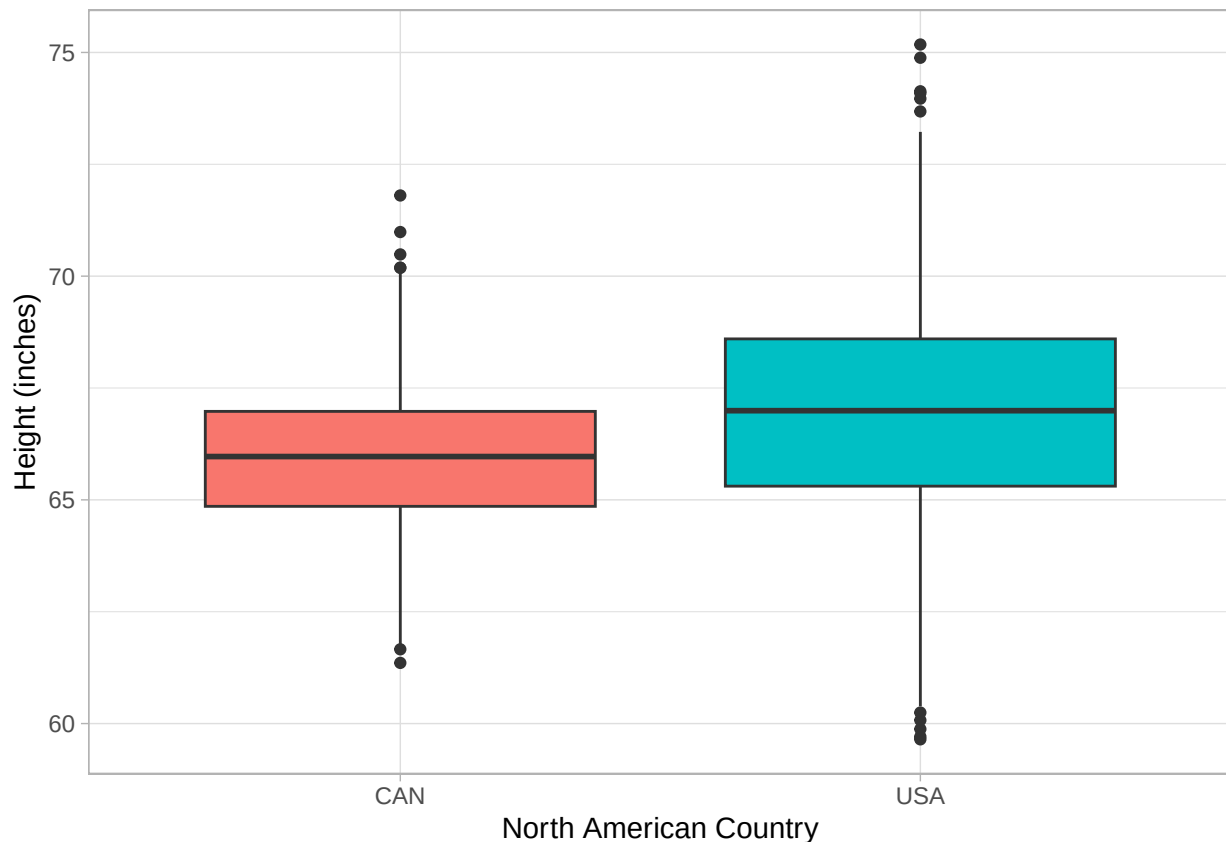
`xlab()` and `ylab()` functions set the labels for the x and y axes, respectively. Simply type the desired label text as a string inside the function.

Additionally, `ggtitle()` adds a title to the plot.

Type `?theme_bw` to explore available themes in `ggtheme {ggplot2}`

The GOAL: Add axis labels and a light theme to the boxplot of height by country.

```
# Code Example: Boxplot with labels and theme
Tall_Box <- Tallness %>%
  ggplot(aes(Country, Height, fill = Country)) +
  geom_boxplot(show.legend = FALSE) +
  xlab("North American Country") +
  ylab("Height (inches)") +
  ggplot2::theme_light()
# Print the boxplot
Tall_Box
```



Comment: `xlab()` and `ylab()` set axis labels. `ggplot2::theme_light()` applies light horizontal grid lines. (The `ggplot2::` prefix explicitly specifies that you're using the `theme_light()` function from the `ggplot2` package. This is useful when multiple packages have functions with the same name.)

9 Summary

In this chapter, you learned that `ggplot2` builds plots by combining a data frame, an aesthetic mapping in `aes()`, and one or more geometric layers such as `geom_point()`, `geom_histogram()`, and `geom_boxplot()`. You practiced starting with a blank canvas, mapping variables to visual properties, and then choosing a geom that fits the question you want to answer.

You also improved plot readability by adding axis labels and applying themes. These are foundational data-storytelling skills in R. You will reuse them whenever you need to explore data or communicate a pattern to someone else.

10 Chapter Terms

Aesthetic mapping: A connection between a variable in the data and a visual property in the plot, such as position, color, or fill. In `ggplot2`, mappings are usually set inside `aes()`.

Boxplot: A graph that summarizes a distribution using the median, quartiles, spread, and possible outliers.

Density Plot: A graph for continuous numerical data that shows a smoothed estimate of the distribution of values. In `ggplot2`, a density plot is typically created with `geom_density()` and can be understood as a smooth alternative to a histogram.

ggplot2: An R package for data visualization based on the grammar of graphics. It builds plots by combining data, mappings, and geometric layers.

geom: A geometric layer that controls how data are drawn on the plot. Examples include `geom_point()`, `geom_histogram()`, and `geom_boxplot()`.

Histogram: A graph that shows the distribution of one numeric variable by grouping values into bins.

Labels: Text added to a `ggplot2` graphic to identify and explain what the plot shows, such as axis labels, legend titles, plot titles, subtitles, captions, and tags. In `ggplot2`, labels are commonly set with `labs()`, `xlab()`, `ylab()`, and `ggtitle()`.

Scatterplot: A graph that shows the relationship between two numeric variables by placing one point for each observation.

Theme: A set of display settings that controls the non-data parts of a `ggplot2` graphic, such as fonts, background, gridlines, axis text, legend appearance, and title styling. Complete themes like `theme_bw()` or `theme_light()` change the overall look of a plot, while `theme()` lets you customize individual parts.

11 Practice Space

In this Practice Space, you are expected to:

- build plots with the three core `ggplot2` steps: `data` → `aes()` → `geom_*()`
- store requested plots with the exact object names given in the task
- use `ggplot2` layers to add labels, color or fill mappings, transparency, and other formatting
- check your results by printing plots and using simple checks such as `class()`, `names()`, `head()`, `nrow()`, `table()`, or other appropriate checks
- write short interpretations that explain what the graph shows, not just what code you ran

When a task asks for explanation or reflection, answer in a short memo after your code and checks.

Replace every `____` placeholder and any `TODO` comment with working code or a short written response. Run each section and make sure the requested objects appear in the Environment. For every task, use the pattern Goal → Code → Checks → Explain.

You will submit your `chapter16_practice.qmd`, your rendered `chapter16_practice.html` (optional), and your `chapter16_practice.RData` workspace file.

11.1 Setup

1. Create a **new Quarto file** in your course folder and save it as:
 - `chapter16_practice.qmd`
 - Test Render your document often so you can catch formatting problems and code errors early.
 - You may also render and submit `chapter16_practice.html`.
2. Use the heading structure taught earlier in the chapter.

Your Quarto setup for Chapter 16 should include headings, section numbering, and a table of contents, and you should render often to check both formatting and errors.

3. Load the packages used in this chapter

This Practice Space uses `ggplot2`, `dplyr`, and `dslabs`, so your document should load those packages near the top and run cleanly from start to finish.

4. Make your plotting work reproducible.

- Run your document from top to bottom to confirm that your plots, objects, and explanations all match.
- Print each plot object after you create it so you can verify what it looks like in your rendered document.
- Make sure the requested plot objects appear in your Environment with the exact names listed in the tasks.

5. Memo your work with the / / /

- a memo before the code chunk that states the goal of the code and what you are trying to learn, check, or demonstrate
- the code chunk itself
- any needed checks after the code
- a memo after the code chunk that explains what you learned, discovered, or confirmed
- answers to all comment questions included in the task

If you **used help** while working, document it in a memo. For example, if you used an LLM or another outside resource to understand an error or concept, briefly disclose that use and explain what help you received. **Type all code yourself.** This is part of the course expectation for AI-supported learning and memo-based workflow.

Follow this template for each coding task.

What is the goal of the code? What am I trying to learn, check, or demonstrate?

```
# Code
your_object <- "here"
```

What checks can I do to confirm that the code worked as expected?

```
# Checks
class(your_object)
your_object
```

/ What I learned / Next step. 1–2 sentences.

11.2 Task 0

Packages

Make sure your script includes code that attached `ggplot2`, `dplyr`, and `dslabs` packages so that your script runs from start to finish without errors.

```
# Code
library(ggplot2)
library(dplyr)
library(dslabs)
```

11.3 Task 1

Quick Recall

List the three core `ggplot2` steps and write a short explanation of each step.

EXPLANATION: “_____”

EXPLANATION: “_____”

EXPLANATION: “_____”

11.4 Task 2

Scatterplot

Create a scatterplot from the built-in `iris` dataset, and save it in an object called `Pretty_Flowers`. Put sepal length on the horizontal axis and petal length on the vertical axis. Color the points by species so that each species has a different color. Make the points moderately large so they are easy to see. Finally, label the x-axis as “Sepal Length (cm)” and the y-axis as “Petal Length (cm)”.

```
# Code
Pretty_Flowers <- iris %>%
  ggplot(aes(x = _____, y = _____, color = _____)) +
  geom_point(size = 3) +
  xlab(_____) +
  ylab(_____)
```

```
# Checks
class(Pretty_Flowers)
pretty_flowers
```

/ What pattern do you notice in petal length across species?

11.5 Task 3

Boxplot: Group Comparison

Create a boxplot from the built-in `iris` dataset and store it in an object called `Petal_Spread`. Put species on the horizontal axis and petal width on the vertical axis. For each species, draw a separate boxplot showing the spread and typical values of petal width. Fill each box with a different color according to species, but remove the legend so only the colored boxplots and axis labels indicate which species is which.

```
# Code
Petal_Spread <- iris %>%
  _____(____ = Species, ____ = Petal.Width, _____ = Species)) +
  _____(_____ = FALSE)
```

```
# Checks
class(Petal_Spread)
Petal_Spread
```

/ Which species appears to have the widest petal-width distribution? What does the boxplot help you see that a table of values would not show as quickly?

11.6 Task 4

Density plot: split + transparency

Create a density plot using the built-in `C02` dataset and store the plot in an object called `C02_Graph`. Put CO2 uptake on the horizontal axis. Use plant type to define separate smooth curves, so each plant type has

its own density curve filled with a different color. Make the filled areas partially transparent so the curves can overlap and still be visible where they intersect. Finally, add the title “CO2 Uptake by Plant Type” (use `ggtitle()`) at the top of the plot.

```
# Code
C02_Graph <- C02 %>%
  _____(x = uptake, fill = Type)) +
  _____(alpha = 0.5) +
  _____(_____ )
C02_Graph

# Checks
class(C02_Graph)
C02_Graph
```

/ What does the overlap between the density curves suggest about the two plant types?

11.7 Task 5

Histogram vs. Density

Create a graph from the `heights` dataset and save it in an object called `Tall_Density`. Put height (in inches) on the horizontal axis. Draw a histogram of heights using bars that are 1 inch wide, filled in blue with white borders. Scale the histogram so that the vertical axis shows density (proportion), not raw counts. On top of this, add a smooth density curve that shows the overall shape of the height distribution, using a dark orange line. Fill the area under that density curve with the same dark orange color, but make it partially transparent so you can still see the blue histogram bars underneath. Label the x-axis “Height (inches)” and the y-axis “Density” (**Hint:** See Layering geoms and use `geom_histogram()` first, then `geom_density()`.)

```
# Code
Tall_Density <- heights %>%
  _____(_____ = height)) +
  _____(_____ = after_stat(density)),
  _____ = 1,
  _____ = "blue",
  _____ = "white") +
  _____(color = "darkorange",
  fill = "darkorange",
  _____ = 0.3,
  _____ = 1) +
  _____("Height (inches)") +
  _____("Density")
```

```
# Checks
class(Tall_Density)
Tall_Density
```

```
head(heights)
nrow(heights)
```

/ What does the density curve add to the histogram? Does the combined graph make the overall shape easier to describe?

11.8 Task 6

Reflect:

Write a short paragraph reflecting on when would a density curve be more useful than a histogram?

EXPLANATION: “_____”

11.9 Save and Upload

- Save your Quarto file as `chapter16_practice.qmd`.
- Save your workspace as `chapter16_practice.RData`.
- You will submit your code file (`.qmd`) and your workspace file (`.RData`).
- You may also submit the rendered HTML file if you can successfully render. Show off your formatting skills!